Sistemas Operativos

Guía de estudio teórica: multiprogramación, MMU, interrupciones y entrada/salida

Esta guía junta toda la teoría que se necesita para entender la práctica de multiprogramación del emulador: qué hace cada componente, por qué existe y cómo se relaciona con los demás. No incluye código; el objetivo es que puedas explicar cada idea con tus palabras.

Contenido

- **Bloque 1.** Hardware base: CPU, memoria, clock, interrupciones, CPU idle y busy
- **Bloque 2.** Memoria y direccionamiento: dirección lógica y física, MMU, batch vs multiprogramación, Loader
- **Bloque 3.** Procesos: estados, PCB, tabla de PCBs, Ready Queue, proceso en ejecución, concurrencia
- **Bloque 4.** Cambio de contexto y Dispatcher
- **Bloque 5.** Entrada/Salida: dispositivos, controlador, cola de espera, #IO_IN y #IO_OUT
- **Bloque 6.** El S.O. como máquina de estados: los 4 handlers y el flujo completo
- **Ejercicio 4.** Qué hace la CPU durante un I/O y cuántos ticks lleva el batch (resuelto)
- **Respuestas a los puntos 1, 2 y 3 de la consigna**, preguntas de autoevaluación y glosario

Cómo está organizada

Los 6 bloques siguen el orden en que se construye un sistema operativo: primero el hardware, después cómo se usa la memoria, después qué es un proceso, cómo se cambia de uno a otro, cómo se maneja el I/O y, al final, cómo se coordina todo con interrupciones.

Bloque 1 · Hardware base

1.1 Los componentes y para qué existen

La arquitectura del emulador separa el trabajo en piezas con un rol claro. Una buena analogía es una fábrica:

Componente	Rol	Analogía
CPU	Ejecuta instrucciones, de a una por tick. No toma decisiones de gestión: solo ejecuta y avisa.	El obrero
Memoria	Guarda las instrucciones de todos los programas cargados.	El depósito
Clock	Marca el ritmo: cada tick le avisa a CPU y dispositivos que avanzaron un ciclo.	El latido / metrónomo
Interrupt Vector	Recibe las interrupciones y las deriva al handler que corresponde a cada tipo.	La recepcionista
MMU	Traduce direcciones lógicas a físicas para que la CPU pueda leer la memoria.	El traductor
Dispositivo de I/O	Hace operaciones lentas (imprimir, leer disco, bajar archivos), una a la vez.	La impresora de la oficina

Idea clave
La CPU es “tonta” a propósito. Solo ejecuta y avisa cuando pasa algo que no puede resolver sola. Quien decide qué hacer con cada aviso es el Sistema Operativo (el software), a través de los handlers de interrupción. Esta separación hace el diseño modular: el hardware no necesita saber nada de políticas del S.O.

1.2 El tick y el ciclo fetch → decode → execute

Cada vez que el Clock da un tick, la CPU (si tiene algo para hacer) ejecuta una instrucción completa en tres pasos:

- **Fetch**: busca en memoria la próxima instrucción, usando el Program Counter (PC), que indica “en qué instrucción voy”. Importante: inmediatamente después del fetch, el PC se incrementa, así que durante la ejecución ya apunta a la instrucción *siguiente*.
- **Decode**: interpreta de qué tipo es (CPU común, IO o EXIT). En el emulador este paso no hace nada visible.
- **Execute**: la ejecuta. Una instrucción de CPU común simplemente consume el tick; IO y EXIT disparan interrupciones.

El Clock avisa a sus suscriptores en un orden fijo. En el emulador el dispositivo de I/O está suscripto antes que la CPU, por lo que **en cada tick el dispositivo avanza primero y la CPU después**. Este detalle explica por qué un proceso que termina su I/O puede volver a correr en el mismo tick en que se avisa que terminó.

1.3 CPU idle y CPU busy

La CPU indica si tiene trabajo con el valor del PC:

- **PC = -1**: CPU **idle** (ociosa). No hay proceso corriendo y el tick no hace nada.
- **PC mayor que -1**: CPU **busy** (ocupada). Hay un proceso corriendo y el PC dice en qué instrucción está.

Un tick con la CPU idle es tiempo desperdiciado. Todo el objetivo de la multiprogramación es evitar esos ticks perdidos.

1.4 Interrupciones (IRQ)

Una interrupción es una señal de hardware que dice “pasó algo que requiere atención del S.O.”. La CPU, cuando se encuentra con una instrucción especial, no la resuelve: arma una solicitud de interrupción (IRQ) con un **tipo** y, opcionalmente, un **parámetro**, y se la entrega al Interrupt Vector. El Interrupt Vector es en esencia una tabla que asocia cada tipo de interrupción con su handler; busca el handler del tipo recibido y lo ejecuta.

Interrupción	Quién la origina	Cuándo	Qué significa
#NEW	El kernel (system call de ejecutar un programa)	Cuando se pide correr un programa	Hay un proceso nuevo para crear
#KILL	La CPU	Al ejecutar la instrucción EXIT	El proceso en CPU terminó
#IO_IN	La CPU	Al ejecutar la instrucción IO	El proceso en CPU necesita ir a I/O
#IO_OUT	El dispositivo de I/O	Cuando termina la operación	El proceso que estaba en I/O ya puede volver

El Interrupt Vector atiende de a una interrupción por vez (usa un candado), para que dos handlers nunca modifiquen las estructuras del S.O. al mismo tiempo.

> **Comprobá que lo entendiste:** ¿por qué decimos que la CPU “no resuelve nada” cuando encuentra un EXIT o un IO, y quién se hace cargo realmente? ¿Qué contiene el PC cuando la CPU está idle?

Bloque 2 · Memoria y direccionamiento

2.1 Programa e instrucciones

Un programa es una lista de instrucciones. Al escribirlo se arma combinando bloques: por ejemplo “4 instrucciones de CPU, un IO, 1 de CPU”. Algunos bloques son listas (varias instrucciones de CPU) y otros son una sola instrucción (el IO). Antes de guardarlo, el programa se **aplana**: las listas se desarman y todas las instrucciones quedan una al lado de la otra, una por celda. Además, si el programa no termina con EXIT, se le agrega uno al final.

Esa operación de “desarmar una lista y agregar sus elementos sueltos” es la diferencia entre agregar una lista como un solo elemento (queda anidada, y la memoria tendría una lista entera en una celda) y agregar cada elemento por separado. La memoria necesita lo segundo: **una instrucción por celda**.

Lo que escribís	Lo que se guarda en memoria
CPU×4, IO, CPU×1	CPU, CPU, CPU, CPU, IO, CPU, EXIT (7 celdas; el EXIT se agrega solo)

2.2 Dirección lógica vs dirección física

Cuando se escribe un programa, sus instrucciones se numeran empezando desde cero: la primera es la 0, la segunda la 1, y así. Esa numeración es relativa al propio programa y no depende de dónde termine guardado. Es como numerar las páginas de un cuaderno del 1 en adelante, sin importar en qué estante de la biblioteca está.

- **Dirección lógica**: posición de la instrucción *dentro del programa* (relativa). Es lo que maneja la CPU: el PC.
- **Dirección física**: posición real de esa instrucción en la memoria principal (absoluta).

Ejemplo de la cátedra: el proceso 3 tiene sus instrucciones en las direcciones lógicas 0, 1, 2 y 3, pero en la memoria real están guardadas en las celdas 5, 6, 7 y 8. La CPU sigue creyendo que el programa empieza en 0.

> **Idea clave**
> Si varios programas conviven en memoria, no pueden empezar todos en la celda 0 física: se pisarían. Por eso cada proceso cree que arranca en 0, pero en realidad cada uno arranca en un lugar distinto. Alguien tiene que traducir de “mi 0” al lugar real, y ese alguien es la MMU.

2.3 La MMU (Memory Management Unit)

La MMU es el componente de hardware que transforma direcciones lógicas en físicas. Responde la pregunta de la consigna: **la CPU no accede directamente a la memoria** porque necesita un traductor en el medio que le permita ubicar las instrucciones de un proceso sin saber dónde está guardado.

La traducción es una suma:

> **dirección física = dirección lógica (PC) + baseDir**

El **baseDir** (dirección base) es la celda física donde empieza el programa del proceso que está corriendo. La MMU lo tiene guardado de antemano: se lo carga el Dispatcher cuando ese proceso pasa a running. Por eso, cuando la CPU hace el fetch, solo le manda el PC (la dirección lógica) y la MMU usa el baseDir que ya tenía almacenado.

Flujo completo del fetch: la CPU pide la instrucción de la posición lógica PC → la MMU le suma el baseDir → lee esa dirección física en la memoria → la memoria devuelve la instrucción → la CPU la recibe.

Además del baseDir, la MMU tiene un **límite**: si la dirección lógica pedida supera el tamaño permitido del proceso, se produce un error de dirección inválida. Esto protege a un proceso de leer la memoria de otro (**protección**), mientras que el baseDir resuelve la **reubicación**.

Ojo
La MMU siempre tiene el baseDir del proceso que está en running. Cada vez que cambia el proceso en la CPU, hay que actualizar el baseDir de la MMU junto con el PC de la CPU. Si se olvida, la CPU leería instrucciones de otro programa.

2.4 Batch vs multiprogramación

- **Batch**: los programas se cargan y ejecutan de a uno. Mientras corre el primero, los otros ni están en memoria.
- **Multiprogramación**: **todos los programas están en memoria al mismo tiempo**, cada uno en su rango de direcciones físicas.

La ventaja: cuando un proceso se va a I/O y no puede usar la CPU, el S.O. pone a correr a otro que ya está cargado, sin esperar a cargarlo. Así la CPU no queda ociosa.

Aclaración importante: estar en memoria no es estar ejecutándose. Hay una sola CPU, así que en cada instante solo un proceso corre. Los demás están cargados pero "pausados": esperando turno (ready) o esperando I/O (waiting). Es como tener tres libros abiertos sobre el escritorio: podés saltar rápido de uno a otro, pero leés uno por vez.

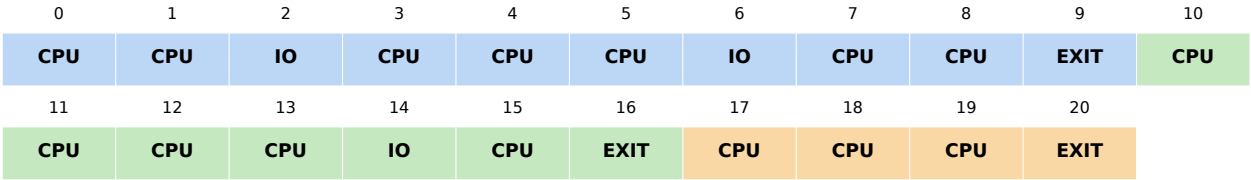
2.5 El Loader

El Loader resuelve "¿en qué baseDir cargo este programa?". Recuerda cuál es la próxima celda libre de la memoria; al cargar un programa lo copia a partir de ahí, devuelve esa celda inicial (su baseDir) y avanza la marca hasta el final de lo que ocupó. Ese baseDir es el que después se guarda en la ficha del proceso (PCB) y usa la MMU.

Ejemplo con los tres programas de la consigna (cada uno incluye su EXIT final):

Programa	Instrucciones	Cantidad	baseDir	Celdas físicas
prg1	CPU×2, IO, CPU×3, IO, CPU×2, EXIT	10	0	0 a 9
prg2	CPU×4, IO, CPU×1, EXIT	7	10	10 a 16
prg3	CPU×3, EXIT	4	17	17 a 20

Así queda la memoria (azul: prg1, verde: prg2, naranja: prg3; el número sobre cada celda es la dirección física):



Con una memoria de 25 celdas sobran 4 (21 a 24). Ejemplos de traducción: la instrucción IO de prg2 está en la dirección lógica 4, así que la física es 4 + 10 = 14. La instrucción EXIT de prg3 está en la lógica 3, la física es 3 + 17 = 20.

Comprobá que lo entendiste: si un proceso tiene baseDir 5 y su PC vale 2, ¿qué dirección física pide la MMU? (Respuesta: 7). ¿Por qué ayuda la multiprogramación a que la CPU esté menos tiempo idle?

Bloque 3 · Procesos

3.1 Programa vs proceso

Un **programa** es código estático: la lista de instrucciones, como un archivo ejecutable. Un **proceso** es ese programa *en ejecución*: tiene un estado, una posición actual (PC), una ubicación en memoria (baseDir) y va cambiando con el tiempo. La relación es la de una receta (programa) y esa receta siendo cocinada en un momento determinado (proceso). Por eso, al pedir ejecutar un programa, el kernel primero lo convierte en un proceso con su propia ficha.

3.2 Estados de un proceso

Estado	Significado
new	Recién pedido; todavía no fue admitido oficialmente por el sistema.
ready	Listo para correr, esperando que se libere la CPU.
running	Está siendo ejecutado por la CPU en este instante (solo uno a la vez).
waiting	Esperando algo para poder seguir; acá, que termine su operación de I/O.
terminated	Ya terminó su ejecución.

Transiciones entre estados y la interrupción que las provoca en el emulador:

Transición	Nombre en el diagrama	Qué la dispara
new → ready	admitted	#NEW: el S.O. crea y admite el proceso
ready → running	scheduler dispatch	El S.O. elige el proceso y el Dispatcher lo carga en la CPU
running → waiting	I/O or event wait	#IO_IN: el proceso pidió I/O
waiting → ready	I/O or event completion	#IO_OUT: el dispositivo terminó
running → terminated	exit	#KILL: el proceso ejecutó EXIT
running → ready	interrupt	Desalojo del proceso (por ejemplo por fin de un quantum de tiempo). En esta práctica no se usa.

Idea clave
El diagrama de estados es el mapa de todo lo que se programa en la práctica: cada interrupción es un "empujón" que mueve a uno o más procesos de un estado a otro. Un proceso nuevo que encuentra la CPU libre pasa directamente a running (conceptualmente atraviesa ready).

3.3 PCB (Process Control Block)

El PCB es la ficha de cada proceso: guarda todo lo que el S.O. necesita para poder "pausarlo" y "reanudarlo" después. Como mínimo contiene:

- **pid**: identificador único del proceso.
- **baseDir**: celda física donde empieza su programa (lo da el Loader).
- **pc**: por qué instrucción lógica va; funciona como marcapáginas.
- **state**: en cuál de los 5 estados está.
- **path**: nombre del programa (por ejemplo "prg1.exe").

El PC del PCB y el PC de la CPU son cosas distintas: el de la CPU es el que se usa en vivo, y el del PCB es una copia guardada. Cuando el proceso sale de la CPU, su PC se copia al PCB para no perder

el progreso; cuando vuelve, se copia del PCB a la CPU. Hacen falta los dos porque la CPU es una sola y la usan muchos procesos por turnos, mientras que cada proceso necesita recordar dónde se quedó.

Esto responde al comentario que figuraba en el código viejo (“¿por qué hacemos esto?”): al dejar la CPU idle el PC se pierde, así que había que guardarlo en algún lado. Esa ficha improvisada era una versión mínima del PCB.

3.4 PCB Table

Es la tabla central que mantiene el kernel con **todos** los PCBs del sistema, sin importar su estado. Permite buscar un PCB por pid, agregar uno nuevo, quitarlo (por ejemplo cuando termina) y generar pids nuevos. Los pids son únicos y **nunca se reutilizan**. Es como un padrón o registro civil de procesos.

3.5 Ready Queue y cola de espera

La **Ready Queue** es una cola FIFO (el primero que entra es el primero que sale) con los PCBs de los procesos en estado ready. Cuando la CPU queda libre, el S.O. saca el primero de esta cola y lo pone a correr. Los procesos en waiting no están en la Ready Queue: esperan en la cola del controlador del dispositivo de I/O (Bloque 5). Y el que está en running tampoco está en ninguna cola: lo referencia el kernel aparte.

3.6 El proceso en ejecución (runningPCB)

El kernel mantiene una referencia directa al PCB que está actualmente en la CPU. Les sirve a los handlers para saber “¿quién estaba corriendo?”: por ejemplo, al llegar un #IO_IN, hay que saber a quién mandar a la cola de espera. Cuando la CPU queda libre, esta referencia debe quedar vacía.

3.7 Concurrencia vs paralelismo

- **Concurrencia**: varios procesos avanzan “a la vez” alternándose en una sola CPU, tan rápido que parece simultáneo. Es lo que hace el emulador.
- **Paralelismo real**: varios procesos ejecutando instrucciones en el mismo instante exacto, lo que requiere varias CPUs o núcleos. El emulador no lo tiene.

3.8 ¿Qué es el kernel?

El kernel es el núcleo del Sistema Operativo: el software con privilegios especiales que gestiona los recursos del sistema (CPU, memoria, dispositivos) y reacciona a las interrupciones para decidir qué hace cada proceso en cada momento. Contiene la tabla de procesos, la Ready Queue, el proceso en ejecución, el Loader, el Dispatcher y los handlers.

> **Comprobá que lo entendiste:** si un proceso está en la Ready Queue y llega un #IO_OUT de otro proceso, ¿el que estaba en Ready cambia de estado? (Respuesta: no; sigue en ready esperando su turno. El que cambia es el que volvió del I/O, que pasa de waiting a ready o a running.)

Bloque 4 · Cambio de contexto

4.1 Context switch

Es la operación de sacar un proceso de la CPU y poner otro en su lugar. Cuando hay una interrupción, el S.O.:

- **Guarda el estado** del proceso que corría en su PCB (en esta CPU, básicamente el PC; en CPUs reales también los registros).
- **Recarga el estado** del proceso que va a entrar, leyéndolo de su PCB.

Durante el cambio, ninguno de los dos procesos avanza: es el costo administrativo de alternar. El context switch es lo que hace posible la concurrencia con una sola CPU; sin él, un proceso que se va a esperar I/O dejaría a la CPU trabada.

4.2 El Dispatcher

El Dispatcher es el componente que hace efectivo el cambio de contexto. Funciona como “driver” de la CPU y la MMU: mueve datos entre el PCB y el hardware. Tiene dos operaciones, una espejo de la otra:

Operación	Qué hace	En términos de datos
Cargar (load) un proceso	Pone al proceso a correr	PC del PCB → CPU; baseDir del PCB → MMU
Guardar (save) un proceso	Lo saca de la CPU y deja la CPU idle	PC de la CPU → PCB; PC de la CPU = -1

Detalle importante: como el PC se incrementa justo después del fetch, cuando la CPU ejecuta un IO o un EXIT su PC ya apunta a la instrucción siguiente. Entonces, al guardar, el PCB recuerda la instrucción posterior al IO, y cuando el proceso vuelva continuará desde ahí sin repetir el IO.

Ejemplo: un proceso ejecuta su IO en la dirección lógica 2. Al terminar el fetch, el PC vale 3. Se guarda 3 en el PCB y la CPU queda idle. Mucho después, al cargarlo de nuevo, la CPU recibe PC = 3 y retoma justo después del IO.

4.3 ¿Qué pasa si no hay nadie para cargar?

Si se guarda un proceso y la Ready Queue está vacía, no hay otro a quien cargar y la CPU **queda idle** (PC = -1), ejecutando “no-operación” tick tras tick hasta que alguien vuelva a estar listo (por ejemplo un proceso que termina su I/O). Es exactamente el tiempo desperdiciado del que hablábamos en el Bloque 1.

Bloque 5 · Entrada/Salida (I/O)

5.1 Qué es un I/O

Una operación de entrada/salida es cualquier intercambio de datos con un dispositivo externo o periférico (imprimir, leer un disco o DVD, bajar algo de internet) que la CPU no puede hacer por sí misma y que, además, tarda mucho más que una instrucción común de CPU.

Dos propiedades de estos dispositivos condicionan todo el diseño:

- Son **lentos** comparados con la CPU.
- Solo pueden hacer **una operación a la vez**.

5.2 El dispositivo y su tiempo

Un dispositivo ejecuta una operación contando ticks. La impresora del emulador tarda 3 ticks. Si recibe una operación estando ocupado, es un error: el S.O. debe garantizar que los pedidos no se choquen. Cuando termina, el dispositivo **avisa** lanzando una interrupción #IO_OUT.

Cronología de un I/O (la CPU ejecuta el IO en el tick t; el dispositivo avanza antes que la CPU en cada tick):

Tick	Qué pasa
t	La CPU ejecuta el IO y lo manda al dispositivo (consume 1 tick). El dispositivo queda ocupado.
t+1, t+2, t+3	El dispositivo cuenta 1, 2 y 3 de 3. Está ocupado.
t+4	El dispositivo supera su tiempo, termina y lanza #IO_OUT. El proceso ya puede volver a correr en ese mismo tick.

Es decir: 1 tick para mandar la operación más 3 ticks de espera en el dispositivo.

5.3 El controlador del dispositivo (IoDeviceController)

Es el componente de software que envuelve al dispositivo físico para resolver el problema de que solo atiende una operación a la vez, pero pueden llegar varios pedidos de distintos procesos. Es el “gerente de turnos”; el dispositivo es el “trabajador”. Funciona así:

- Cada pedido llega con el proceso que lo hizo y la operación, y se agrega al final de una **cola de espera**.
- Si el dispositivo está libre, se saca el primer pedido de la cola, se anota qué proceso está siendo atendido y se le manda la operación al dispositivo.
- Si está ocupado, el pedido espera en la cola.
- Cuando el dispositivo termina, el controlador devuelve el proceso que acaba de ser atendido, queda sin proceso en curso y, si hay más pedidos en la cola, manda el siguiente al dispositivo.

Es igual a una impresora de oficina: si tres personas mandan a imprimir juntas, se imprime una por una, en orden de llegada.

PrinterIODevice es la implementación concreta de un dispositivo (una impresora simulada con un tiempo de 3 ticks). El controlador no depende de qué dispositivo sea: solo necesita saber si está libre y mandarle operaciones.

5.4 Los procesos en I/O: dónde esperan y en qué estado

Si tres procesos piden I/O casi al mismo tiempo, uno es atendido por el dispositivo y los otros dos esperan en la cola del controlador. **Los tres están en estado waiting**, tanto el que está siendo atendido como los que esperan turno: ninguno puede usar la CPU hasta que termine su propia operación.

5.5 Las interrupciones #IO_IN y #IO_OUT

- **#IO_IN** nace en la **CPU**, por una instrucción del programa (IO). Significa “este proceso necesita I/O”. La CPU no ejecuta la operación: lanza la interrupción.
- **#IO_OUT** nace en el **dispositivo**, cuando termina. Significa “este proceso terminó su I/O y está listo para competir otra vez por la CPU”.

> **Idea clave**
> Asimetría: #IO_IN la origina el programa (a través de la CPU) y #IO_OUT la origina el hardware del dispositivo. Por eso el diagrama de hardware muestra dos flechas de interrupción distintas hacia el Interrupt Vector.

5.6 Punto 3 de la consigna: cómo se llega a ejecutar cada handler

Handler de #IO_IN:

- El proceso corre en la CPU, y en un tick la CPU hace fetch de una instrucción IO.
- Al ejecutarla, la CPU reconoce que es de tipo IO, arma una IRQ de tipo #IO_IN (con la instrucción como parámetro) y se la entrega al Interrupt Vector.
- El Interrupt Vector busca en su tabla el handler registrado para #IO_IN y lo ejecuta.

Handler de #IO_OUT:

- El clock avisa del tick al dispositivo, que está ocupado y va contando.
- Cuando su cuenta supera su tiempo de operación, el dispositivo se marca libre, arma una IRQ de tipo #IO_OUT y se la entrega al Interrupt Vector.
- El Interrupt Vector busca el handler registrado para #IO_OUT y lo ejecuta.

Lo que hace posible todo esto es que, al arrancar, el kernel **registra** cada handler en el Interrupt Vector asociándolo al tipo de interrupción. Sin ese registro, el Interrupt Vector no sabría a quién llamar.

Bloque 6 · El S.O. como máquina de estados

6.1 La idea central

El Sistema Operativo no tiene un bucle propio corriendo todo el tiempo: es código que se activa **solamente cuando llega una interrupción**. Algo pasa (una IRQ), el S.O. reacciona y vuelve a quedar “dormido” hasta la próxima. Es una máquina de estados: cada interrupción mueve procesos entre los estados del Bloque 3, usando las piezas de los Bloques 2 a 5.

6.2 Los 4 handlers

#NEW (se crea un proceso)

- El Loader carga el programa en memoria y devuelve su baseDir.
- Se crea un PCB nuevo (pid único, baseDir, PC en 0, estado new) y se agrega a la PCB Table.
- Si la CPU está libre: el proceso pasa a running y el Dispatcher lo carga.
- Si la CPU está ocupada: el proceso pasa a ready y se encola en la Ready Queue.

#IO_IN (un proceso pide I/O)

- El Dispatcher guarda el proceso que corría (se guarda su PC y la CPU queda idle).
- El proceso pasa a waiting y su pedido se entrega al controlador del dispositivo.
- Como la CPU quedó libre, se busca reemplazo: si hay alguien en la Ready Queue, se lo saca, pasa a running y se lo carga.

#IO_OUT (un proceso terminó su I/O)

- Se le pide al controlador el proceso que acaba de terminar.
- Pasa de waiting a ready.
- Si la CPU está libre, se lo carga y pasa a running; si está ocupada, se lo encola en la Ready Queue.

#KILL (un proceso terminó)

- El proceso en ejecución pasa a terminated y la CPU queda libre (se puede, además, quitar su PCB de la tabla).
- Como la CPU quedó libre, se busca reemplazo en la Ready Queue, igual que en #IO_IN.

6.3 El patrón que se repite

Handler	Situación	Qué hay que hacer
#IO_IN y #KILL	La CPU **se libera**	Sacar al proceso que se va y **buscar reemplazo** en la Ready Queue
#NEW y #IO_OUT	Aparece un proceso **disponible**	**Chequear si la CPU está libre**: si lo está, correrlo; si no, a la Ready Queue

Como estas dos mitades se repiten, conviene concentrarlas en dos operaciones auxiliares reutilizables: “si hay alguien en ready, cargarlo en la CPU” y “correr este proceso si la CPU está libre; si no, encolarlo”.

> **Ojo**
> Orden en #KILL e #IO_IN: primero hay que sacar al proceso que se va de la CPU (guardar su estado y dejar vacío el “proceso en ejecución”) y recién después buscar el reemplazo. Si se invierte, se pisa el estado del proceso que se está yendo.

6.4 El flujo completo integrado

Todo junto, el recorrido de un proceso es este:

- El kernel recibe el pedido de ejecutar un programa y lanza una interrupción #NEW.
- El handler de #NEW usa el Loader y la PCB Table, y el proceso entra a running (si la CPU estaba libre) o a la Ready Queue.
- Mientras corre, si ejecuta un IO, se lanza #IO_IN: el handler lo saca de la CPU y lo manda al controlador del dispositivo (waiting). Otro proceso ocupa la CPU.
- El dispositivo atiende los pedidos de a uno. Cuando termina el del proceso, lanza #IO_OUT: el proceso vuelve a ready (o a running si la CPU estaba libre).
- Cuando el proceso ejecuta EXIT, se lanza #KILL: termina y otro proceso ocupa la CPU.

La CPU y el dispositivo reciben ticks del Clock; las interrupciones salen de ambos hacia el Interrupt Vector, que las deriva a los handlers; los handlers usan el Loader, la PCB Table, la Ready Queue, el Dispatcher y el controlador de I/O.

6.5 Cómo se reparte el trabajo

Pieza	Naturaleza	Rol
Loader, Dispatcher, PCB, PCB Table, Ready Queue, proceso en ejecución, #NEW como interrupción	Infraestructura (base)	Son las herramientas que los handlers usan
Lógica de #NEW, #IO_IN, #IO_OUT y #KILL	Políticas de interrupciones	Combinan las herramientas para mover procesos entre estados